🏷️ Trazabilidad con Decorador

Descripción

El decorador `@tracking_genai` permite instrumentar funciones (síncronas o asíncronas) registrando su ejecución dentro de un Span centralizado de OpenTelemetry. Este enfoque es preciso y adecuado cuando se desea focalizar la observabilidad en componentes críticos de la arquitectura.

La inclusión del decorador extrae explícitamente las variables de entrada, salidas, posibles excepciones, atributos configurados y latencias, para luego inyectar la información al proveedor.

⚙️ Configuración del Contexto de Trazabilidad: `carrier_mode`

El decorador `@tracking_genai` incluye un parámetro avanzado llamado `carrier_mode`, que define cómo se debe propagar o recuperar el contexto del Span padre a través de una propiedad inyectada en el objeto (normalmente `self._otel_propagation_carrier`).

Existen 3 modos admitidos:

- `"fallback"` **(Por defecto):** El decorador intentará extraer el contexto del objeto inyectado SOLO en caso de que el contexto actual de OpenTelemetry se haya perdido o sea inválido. Es el modo más seguro porque respeta el flujo natural de OpenTelemetry y actúa como una red de seguridad en arquitecturas asíncronas complejas.
- `"force"`**:** Sobrescribe cualquier contexto activo en la ejecución actual y obliga a que el nuevo Span se adjunte estrictamente al contexto inyectado en la instancia del objeto. Esto es fundamental para componentes autónomos (ej. agentes) que necesitan vincular sus logs a un hilo conversacional maestro específico sin importar desde dónde son ejecutados.
- `"off"`**:** Desactiva la extracción inyectada del contexto. El decorador dependerá puramente del contexto activo proporcionado por OpenTelemetry.

🚀 Implementación del Decorador

A continuación, se presentan cuatro esquemas representativos de cómo aplicar `@tracking_genai` en diferentes capas de la aplicación.

Ejemplo 1: Trazabilidad Global de un Caso de Uso (End-to-End)

El decorador se aplica sobre el punto de entrada principal del UseCase para medir todo su ciclo de vida y alcance.

```
from app import telemetry_singleton as telemetry
from monitoria.tracing.events import tracking_genai

# Asegurar la inicialización global en la aplicación
telemetry.setup_init_telemetry()
```

```python
class HandleMessageUseCase(MessageUseCase):

    # La implementación del decorador define explícitamente el 'scope'
    @tracking_genai(
        server_host=telemetry.server_host,
        session_id=telemetry.session_id,
        tracer=telemetry.tracer,
        operation_type='e2e',
        service_type='credicorp-dispatcher',
        capture_inputs=True
    )
    async def execute(
        self,
        message: str,
        session_id: UUID,
        context=None,
        messages=None,
        session_state=None,
        previous_workflow_id=None,
        decision=None,
        trace=None
    ):
        self.conversation_manager.get_or_create_context(session_id)

        user_message = Message(role="user", content=message)
        self.conversation_manager.add_message(session_id, user_message)

        await self.create_workflow(message, session_id)

        response = await self.orchestrator.process_message(...)

        assistant_message = Message(
            role="assistant", content=response.message, metadata=response.metadata
        )
        self.conversation_manager.add_message(session_id, assistant_message)

        return response
```

Ejemplo 2: Creación de un Agente Inyectando Contexto

Cuando debe mantenerse la consistencia contextual del tracing a través de dependencias internas e externas, es imperativo asegurar la correlación utilizando la técnica de inyección del carrier.

```python
from app import telemetry_singleton as telemetry
from monitoria.tracing.events import tracking_genai
from opentelemetry.propagate import inject

telemetry.setup_init_telemetry()

@tracking_genai(
```

```
    server_host=telemetry.server_host,
    session_id=telemetry.session_id,
    tracer=telemetry.tracer,
    operation_type='agent-framework',
    service_type='credicorp-dispatcher',
    capture_inputs=True,
)
def agent_framework_agent(self, llm_setting, conversation_id: str,
chat_message_store_factory):
    agent_name = self.get_agent_name()
    chat_client = AzureOpenAIChatClient(...)

    agent = MonitoredChatAgent(...)

    # Inicialización del carrier y su inyección en la propiedad privada.
    # OpenTelemetry leerá subsecuentemente la correlación inyectada a la
instancia.
    carrier = {}
    inject(carrier)
    agent._otel_propagation_carrier = carrier

    return agent
```

Ejemplo 3: Observabilidad en un Agente Genérico

Para monitorear el desempeño dentro de una implementación de control de Logs de un agente, se utiliza el decorador y se fuerza la retención del enlace (`carrier_mode="force"`).

```
import time
import logging
from agent_framework import AgentRunResponse, ChatAgent
from app import telemetry_singleton as telemetry
from monitoria.tracing.events import tracking_genai

telemetry.setup_init_telemetry()

class LoggingChatAgent(ChatAgent):
    def __init__(self, *args, log_level: int = logging.INFO, **kwargs):
        super().__init__(*args, **kwargs)
        self.logger = logging.getLogger(f"AgentFramework.{self.name or self.id}")
        self.log_level = log_level

    @tracking_genai(
        server_host=telemetry.server_host,
        session_id=telemetry.session_id,
        tracer=telemetry.tracer,
        operation_type='agent-logging-run',
        service_type='credicorp-dispatcher',
        capture_inputs=True,
        carrier_mode="force",
```

```
    )
    async def run(self, *args, **kwargs) -> AgentRunResponse:
        start_time = time.time()

        self._log_input(args, kwargs)

        agent_response = await super().run(*args, **kwargs)
        total_time = time.time() - start_time

        self._log_output(total_time, agent_response)

        return agent_response
```